

Generating and Solving 3D Tetracube Puzzles in Arbitrary Non-Rectangular Volumes with ASP

Abdelhadi Agourzam ✉

Université de Mons (UMONS), Faculté des Sciences, Belgium

Mohamed El ismaily

Université de Mons (UMONS), Faculté des Sciences, Belgium

Abstract

We design and implement a *shape-agnostic, scalable* system for generating and solving 3D tetracube puzzles using Answer Set Programming (ASP). The 8 standard tetracubes (I, T, L, Pyramid, O, N, Z, Z-mirror) are packed into *any* connected 3D volume by simply declaring its cells — no shape-specific solver code is needed. A single integer constant `num_copies = N` scales the puzzle to $8N$ pieces filling $32N$ cells, from the classical 32-cell problem ($N = 1$) up to a 224-cell configuration ($N = 7$). We demonstrate solutions for three non-rectangular shapes (T-shape, L-shape, Plus/Cross) and randomly generated polycubes at multiple scales, with empirical solve times ranging from 30 ms ($N = 1$) to 6 s ($N = 7$) on the slab family. We further show that, without the slab structure, freely-grown 3D volumes are almost never packable, evidencing a genuinely two-sided, conjectured-NP-hard general problem. An interactive 3D visualiser, an independent 6-check verifier, and a random puzzle generator complete the system. All source code is available on GitHub.

2012 ACM Subject Classification Computing methodologies → Knowledge representation and reasoning; Theory of computation → Constraint and logic programming

Keywords and phrases Answer Set Programming, Tetracubes, 3D Puzzle, Non-Rectangular Volumes, Shape-Agnostic, Scalable, Clingo, Exact Cover

Digital Object Identifier 10.4230/LIPIcs.KRR.2026.1

Supplementary Material Source code: <https://github.com/oscarRickovic/KRR2026>

Acknowledgements We thank Jef Wijsen for supervising this project.

1 Introduction

A **tetracube** is a three-dimensional solid composed of four unit cubes joined face-to-face. There are exactly 8 distinct one-sided tetracube types. Together they cover $8 \times 4 = 32$ unit cubes, suggesting the classical puzzle: *can all 8 pieces be packed into a given 3D volume with no gaps and no overlaps?*

Existing work focuses on rectangular prisms ($2 \times 4 \times 4$, $2 \times 2 \times 8$, etc.). This project asks a more general question:

*Can the same ASP engine solve the puzzle for **any** connected 3D volume, and for **any** number of copies of the 8 pieces?*

The answer is yes — in the sense that one unmodified engine *accepts* any such volume as input and returns a correct verdict (a packing, or a proof that none exists). This is a statement about the generality of the engine, not a claim that every volume is packable: as Section 7 shows, most freely-shaped 3D volumes admit no packing at all. The generality rests on two design choices:

1. **Shape abstraction.** The target volume is described entirely by a set of `cell(X,Y,Z)` facts. The puzzle engine never references specific coordinates — it only enforces boundary, non-overlap, and full coverage relative to whichever cells are declared.



© Abdelhadi Agourzam and Mohamed El ismaily;
licensed under Creative Commons License CC-BY 4.0
Knowledge Representation and Reasoning (KRR 2026).

Leibniz International Proceedings in Informatics

LIPICs Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany

2. **Copy scaling.** A single constant `num_copies = N` multiplies the piece count to $8N$ and the required cell count to $32N$. Shapes accommodate this by extending their depth to $2N$ layers.

1.1 Contributions

- C1. **Shape-agnostic ASP engine.** Three non-rectangular configurations (T, L, Plus/Cross) and arbitrary random polycubes are all solved by the same unmodified `puzzle.lp`.
- C2. **Scalable `num_copies` architecture.** Changing N from 1 to 7 scales the problem from 32 to 224 cells without touching any solver code.
- C3. **Automated rotation generation.** All 86 distinct tetracube orientations are derived programmatically from 24 rotation matrices.
- C4. **Configurable puzzle generation.** A hint system with random seed produces puzzle instances at six difficulty levels for any N .
- C5. **Independent verification.** A 6-check Python verifier confirms solution correctness without trusting the solver output.
- C6. **Interactive 3D visualiser.** A matplotlib viewer with Puzzle/Solution toggle adapts automatically to any N .
- C7. **Empirical complexity study.** Solve times measured for $N = 1$ to $N = 7$ reveal a polynomial growth law $\sim N^{2.5}$ on the slab family, contrasted with the hard general case.

2 The Eight Tetracube Pieces

Figure 1 shows all 8 tetracubes in 3D. Each piece is a distinct connected solid of 4 unit cubes. Their rotation counts (3 to 24) reflect their symmetry: high-symmetry pieces (I, O) have few orientations; the fully asymmetric L piece has 24. The Pyramid is the only non-planar piece — it cannot be flattened into a single layer.

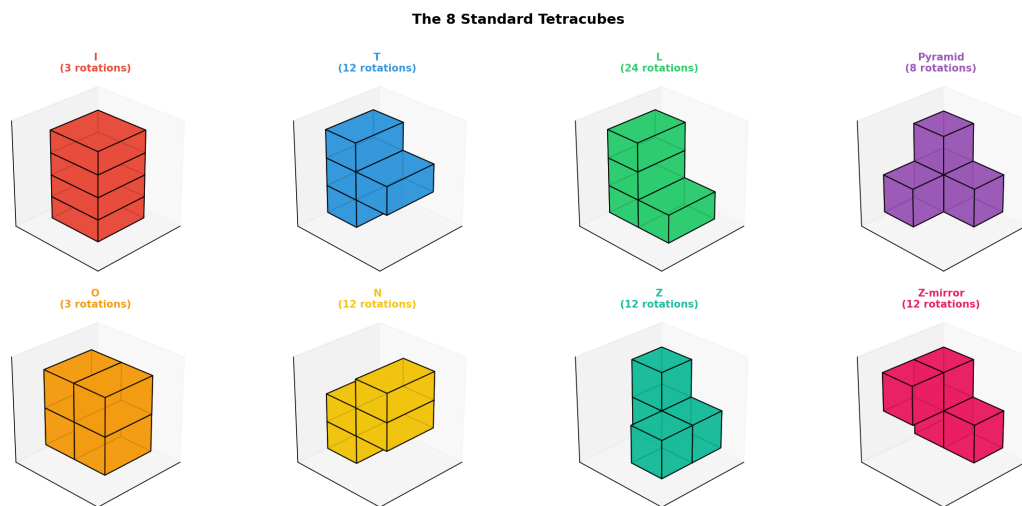


Figure 1 All 8 standard tetracubes rendered in 3D, shown in their base orientation. Each piece is coloured distinctly and labelled with its rotation count. The Pyramid is the only piece that genuinely occupies three spatial dimensions.

Table 1 Properties of the 8 standard tetracubes.

Piece	Rotations	Type	Geometric property
I	3	Planar	Straight line along one axis; 3-fold axis symmetry.
T	12	Planar	T-branch; central cube has one perpendicular neighbour.
L	24	Planar	Hook; chiral and fully asymmetric, hence 24 orientations.
Pyramid	8	3D	3 cubes on base + 1 elevated; only genuinely 3D piece.
O	3	Planar	2×2 square slab; highest symmetry of all pieces.
N	12	Planar	Zigzag; 2 offset rows of 2.
Z	12	Planar	S-shape (right-handed); distinct from its mirror.
Z-mirror	12	Planar	Mirror image of Z (left-handed).
Total	86	7P + 1 3D	$8 \times 4 = 32$ unit cubes.

3 Volume Configurations

3.1 Three Non-Rectangular Shapes

We define three non-rectangular 3D volumes, each with a 16-cell XY footprint extruded to depth 2 (32 cells at $N = 1$). Figure 2 shows their 3D structure.

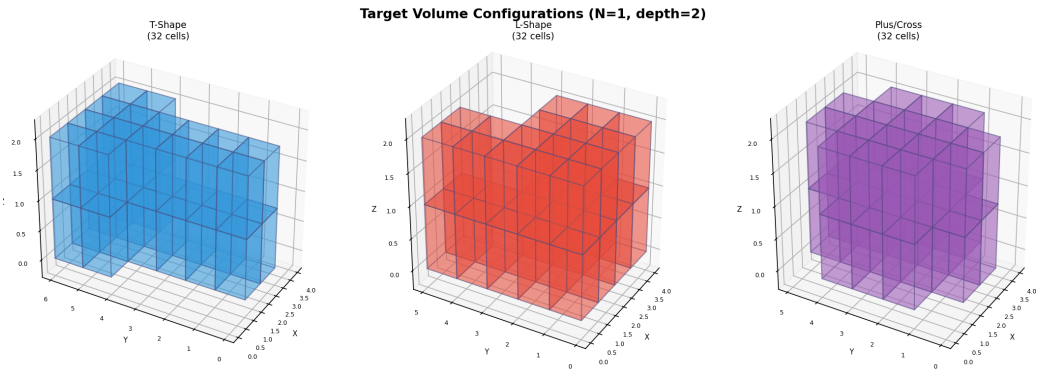


Figure 2 The three target volume configurations rendered in 3D (ghost view, $N = 1$). Left: T-Shape (bar + stem). Centre: L-Shape (rectangle minus top-right corner). Right: Plus/Cross (central body + top and bottom arms). All contain exactly 32 cells.

T-Shape. A wide horizontal bar (4×2 per layer) joined to a narrow stem (2×4 per layer), with no overlap:

$$\text{T-shape} = (\{0..3\} \times \{4, 5\}) \cup (\{1, 2\} \times \{0..3\}) \times \{0, 1\} \longrightarrow 32 \text{ cells.}$$

L-Shape. A 4×5 rectangle per layer with the top-right 2×2 corner removed — a staircase profile:

$$\text{L-layer} = [0..3] \times [0..4] \setminus \{x \geq 2, y \geq 3\} \longrightarrow 16 \text{ cells/layer.}$$

Plus/Cross. A symmetric plus shape: 4×3 central body plus 2×1 arms at top and

1:4 Non-Rectangular Tetracube Puzzles with ASP

bottom:

$$\text{Plus-layer} = \{0..3\} \times \{1..3\} \cup \{1,2\} \times \{4\} \cup \{1,2\} \times \{0\} \longrightarrow 16 \text{ cells/layer.}$$

3.2 Scaling with num_copies

The `num_copies = N` parameter extends each shape's depth to $2N$ layers, giving exactly $32N$ cells for $8N$ pieces. Geometrically, the XY footprint is preserved while the Z-dimension doubles per copy.

$$N \text{ copies} \longrightarrow \underbrace{8N}_{\text{pieces}} \times 4 \text{ cells} = 32N \text{ unit cells} \longrightarrow \text{depth } 2N \text{ layers}$$

Figure 3 compares the T-shape and L-shape at $N = 1$ (depth 2) and $N = 2$ (depth 4), illustrating how the volume grows purely in the Z direction.

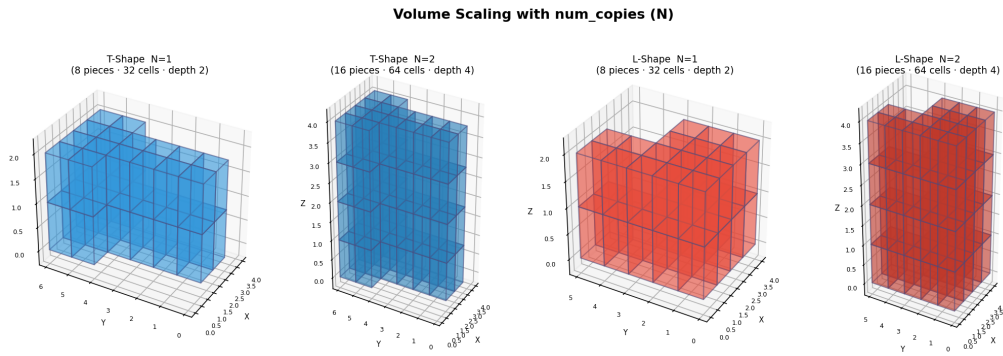


Figure 3 Volume scaling with `num_copies`. The XY footprint is identical for $N = 1$ (left) and $N = 2$ (right); only the depth doubles. T-Shape: 32 cells at $N = 1$, 64 cells at $N = 2$. L-Shape: same. No shape-specific code changes.

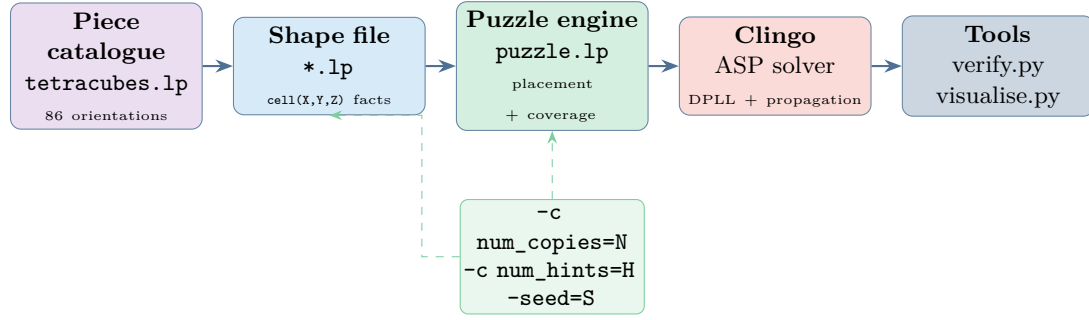
Table 2 Puzzle scale for `num_copies` $N = 1$ to 7.

N	Pieces	Cells	Depth (z)	Overlap pairs to check
1	8	32	0–1	28
2	16	64	0–3	120
3	24	96	0–5	276
4	32	128	0–7	496
5	40	160	0–9	780
6	48	192	0–11	1128
7	56	224	0–13	1540

4 System Architecture

4.1 Pipeline Overview

The system consists of five components that interact in a linear pipeline.



■ **Figure 4** End-to-end pipeline. The shape file and `num_copies` are the only inputs that change between configurations; all other components are reused.

4.2 The `cell/3` Abstraction

The core design insight is that the puzzle engine operates exclusively on the `cell(X,Y,Z)` predicate, never on specific coordinates. Adding a new volume requires only a new file listing its cells.

The three fundamental constraints are:

- (1) **Boundary.** Every cube of every placed piece must land in a declared cell.
- (2) **Non-overlap.** No two distinct pieces may occupy the same cell.
- (3) **Full coverage.** Every declared cell must be occupied by exactly one piece.

Together these three constraints constitute an *exact cover* problem over the cell set. Clingo’s DPLL-based propagation engine solves it efficiently.

4.3 The `num_copies` Scaling Mechanism

The `num_copies` parameter generalises the piece count without any manual enumeration. Piece IDs are assigned types through a cyclic formula: piece P receives the type at position $((P-1) \bmod 8) + 1$ in the 8-type sequence. This ensures every copy of each type is represented exactly once:

- Pieces 1–8 (copy 1): I, T, L, Pyramid, O, N, Z, Z-mirror
- Pieces 9–16 (copy 2): same types, fresh IDs
- Pieces $8(k-1)+1$ to $8k$ (copy k): same cycle

The shape files extend their Z-range to $0..2N-1$, providing exactly $32N$ cells. The solver sees $8N$ pieces that must cover $32N$ cells exactly — the scale changes but the logical structure is identical at every N .

4.4 Hint System and Puzzle Generation

A `num_hints` parameter selects a random subset of pieces to reveal as “given” positions. The remaining pieces become the puzzle to solve. Six difficulty levels arise from varying the hint count from 7 (only 1 hidden piece) to 0 (all $8N$ pieces hidden):

115

Difficulty	Hints shown	Pieces to find ($N = 1$)
Very Easy	7	1
Easy	5–6	2–3
Medium	3–4	4–5
Hard	1–2	6–7
Harder	0	8

116

4.5 File Organisation

■ **Table 3** Project files and their roles (full source on GitHub).

File	Role
<code>tetracubes.lp</code>	86 rotation offsets for all 8 piece types
<code>puzzle.lp</code>	Shape-agnostic placement, coverage, hint logic; <code>num_copies</code>
<code>t_shape.lp</code>	T-shape cells (32 <i>N</i> total, depth 2 <i>N</i>)
<code>l_shape.lp</code>	L-shape cells (32 <i>N</i> total, depth 2 <i>N</i>)
<code>general_shape.lp</code>	Plus/Cross cells — template for any shape
<code>verify.py</code>	Independent 6-check verifier; auto-detects <i>N</i>
<code>visualise.py</code>	Interactive 3D viewer; auto-detects <i>N</i>
<code>random_puzzle.py</code>	Random shape generator with <code>-copies N</code> flag
<code>benchmark.py</code>	Timing comparison T vs L at multiple <i>N</i> values
<code>benchmark_3d.py</code>	Timing for non-decomposable free 3D polycubes

117

5 Solutions

118

The three non-rectangular volumes all admit valid tetracube packings, confirmed by Clingo and independently verified.

119

120

5.1 T-Shape Solution ($N = 1$, 32 cells, 8 pieces)

121

This is the canonical instance: the 8 distinct tetracubes are placed exactly once to fill all 32 cells. Note in particular how the Pyramid — the only genuinely 3D piece — must interlock across both *z*-layers with the surrounding planar pieces, which is what makes even this smallest case non-trivial to pack by hand. Figure 5 shows the result returned by Clingo.

122

123

124

125

5.2 L-Shape Solution ($N = 1$, 32 cells, 8 pieces)

126

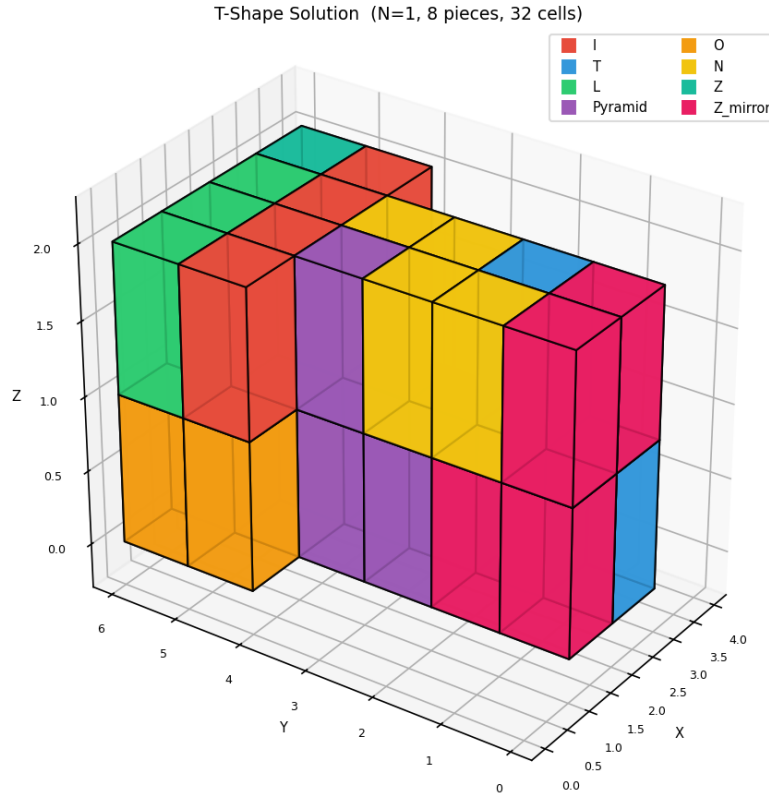
The L-shape replaces the T’s symmetric stem with a staircase profile (the top-right 2×2 corner is removed). Because the boundary is no longer symmetric, the long pieces (I, N, Z) can no longer be placed as in the T-shape and must take different orientations to follow the step. No change to the encoding is required — only the `cell/3` facts differ — yet the same engine still finds a valid packing (Figure 6).

127

128

129

130



■ **Figure 5** T-shape packed solution ($N = 1$). The semi-transparent ghost shows the volume boundary; each coloured solid is a distinct tetracube. Every cell is covered exactly once — no gaps, no overlaps.

5.3 Plus/Cross Solution ($N = 1$, 32 cells, 8 pieces)

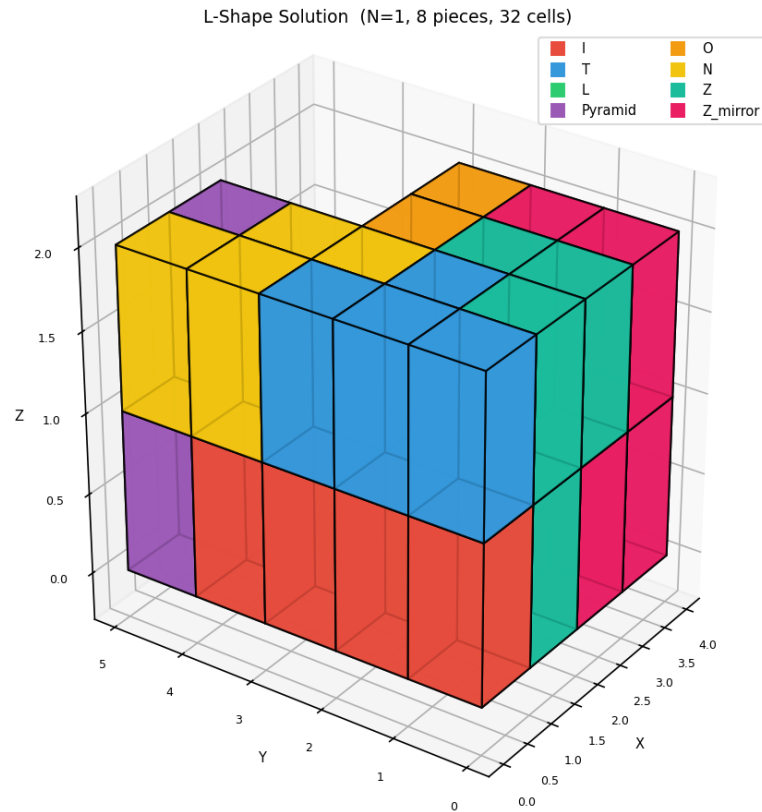
The Plus/Cross is the most geometrically constrained of the three volumes. Its four protruding arms are only two cells wide, so any piece covering an arm has very few legal placements: it must lie flat along the arm and connect back to the central body without spilling outside the boundary. These narrow corner-less regions are what make the shape the hardest of the three to fill, yet a complete packing still exists (Figure 7).

5.4 T-Shape Solution ($N = 2$, 64 cells, 16 pieces)

Setting `num_copies = 2` doubles the volume to depth 4 (64 cells) and the piece set to 16 — two copies of each tetracube type. This is the first multi-copy instance, and it exercises the cyclic type-assignment mechanism of Section 4.3 without any change to the solver. As discussed later in Section 7.4, this stacked-slab volume is in fact decomposable; here we simply note that the engine returns a correct 16-piece packing (Figure 8).

5.5 Random Polycube Puzzles

The three shapes above are hand-designed, so one could suspect they were chosen to suit the solver. *Randomly generated* volumes remove that suspicion and are therefore the decisive test of shape-agnosticism: the engine is handed shapes that neither author has ever seen, with



■ **Figure 6** L-shape packed solution ($N = 1$). The staircase boundary is fully covered. The asymmetric boundary forces different piece orientations than the T-shape, yet the same engine finds a valid packing.

no hand-tuning of the encoding. The `random_puzzle.py` script supplies them and confirms that the *same* unmodified `puzzle.lp` solves them.

149 Generation (Eden model).

A volume is grown one cell at a time: starting from a single seed cell, we repeatedly pick a uniformly random empty face-neighbour of the current cluster and add it. This is the Eden growth model, and it produces connected but otherwise unbiased, irregular polycubes. Two modes are provided:

- **Extruded (`-mode 2d`, default).** Grow a random 16-cell 2D polyomino, then extrude it to depth $2N$. The footprint is arbitrary but every layer is identical, so the volume is a *slab*; combined with `-copies N` this gives $32N$ cells for $8N$ pieces. These instances are reliably solvable (see Section 7.4 for why slabs always admit a packing once the base does).
- **Free 3D (`-mode 3d`).** Grow a connected $32N$ -cell polycube directly in all three axes. Layers differ, so no extrusion or stacking structure exists — the hardest, most general case (analysed empirically in Section 7.5).

The generator retries with fresh seeds until it finds a solvable instance, so a single command produces a complete random puzzle (shape, hints, and solution) for any N . Figure 9 shows three solved extruded instances.

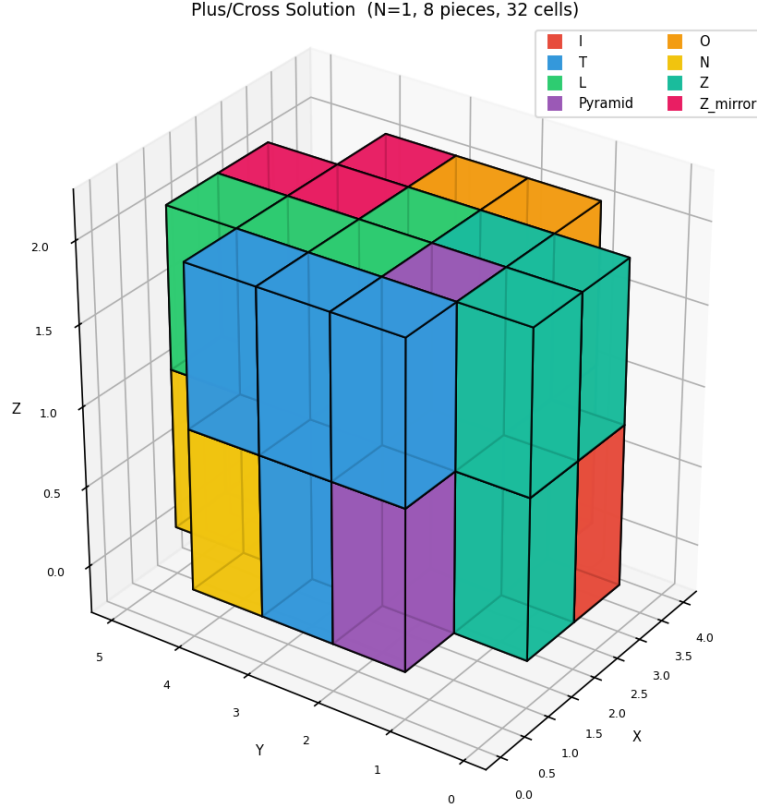


Figure 7 Plus/Cross packed solution ($N = 1$). The non-convex boundary with four arms is entirely covered. This is the most geometrically constrained of the three shapes — the solver must fill the corner-less arms with appropriately oriented pieces.

Random shapes thus play a double role in this work. In the *extruded* mode they are the cleanest demonstration that the engine is truly shape-agnostic — arbitrary, unseen footprints are packed with no code change, and at any scale N . In the *free 3D* mode they become the experimental probe for the general problem: as Section 7.5 shows, such volumes are almost never packable, which is precisely what reveals the genuine difficulty of tetracube packing once the convenient slab structure is removed.

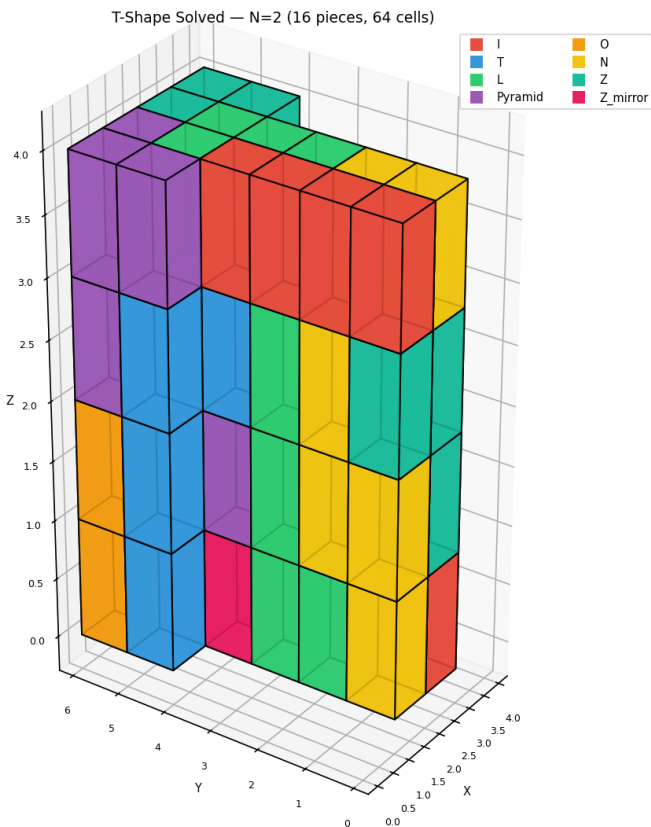
6 Independent Verification

An independent Python verifier (`verify.py`) re-derives the solution from scratch and applies six checks without trusting Clingo. The verifier auto-detects N from the `numCopies` atom in the answer set.

7 Complexity Analysis

7.1 Problem Complexity

NP membership. Tetracube packing is in **NP**: given a placement for all $8N$ pieces, correctness can be verified in polynomial time by checking boundary ($O(N)$), non-overlap ($O(N^2)$ pairs), and coverage ($O(N)$). This is exactly what `verify.py` does.



■ **Figure 8** T-shape packed solution ($N = 2$): two stacked T-slabs forming a 64-cell volume covered by 16 tetracubes. The same solver algorithm scales from $N = 1$ to $N = 2$ with no code changes.

■ **Table 4** Verification results for T-shape and L-shape at $N = 1, 2, 3$. All configurations pass all six checks.

Check		$N = 1$		$N = 2$		$N = 3$	
		T	L	T	L	T	L
1	Correct piece count & type distribution	✓	✓	✓	✓	✓	✓
2	Rotation IDs within valid range per type	✓	✓	✓	✓	✓	✓
3	All piece cells within declared cell set	✓	✓	✓	✓	✓	✓
4	No two pieces share a cell	✓	✓	✓	✓	✓	✓
5	All $32N$ cells covered	✓	✓	✓	✓	✓	✓
6	Hint IDs are a subset of placed IDs	✓	✓	✓	✓	✓	✓
Verdict		PASS		PASS		PASS	

Random 32-Cell Polycube Puzzles (Extruded 2D, depth=2)

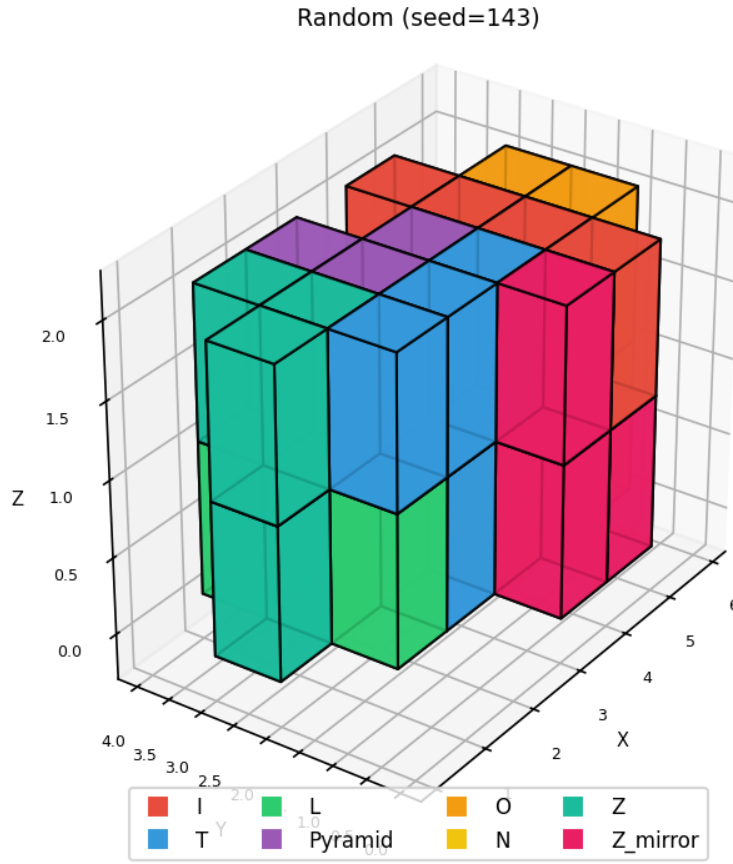


Figure 9 Three randomly generated 32-cell polycube puzzles and their solutions. Each shape was grown by the Eden model with a different random seed. Despite their irregular geometry, all three are solved by the same unmodified puzzle engine in under 50 ms.

The decision problem. The “answer is yes” of Section 1 concerns the *generality of the engine*, not satisfiability. The associated decision problem is genuinely two-sided:

given a connected polycube V with $|V| = 32k$, can V be partitioned into exactly k copies of each of the 8 tetracube types (rotations allowed)?

This problem has both yes- and no-instances — indeed, most freely-grown 3D polycubes are no-instances (Section 7.4, Table 7). The T-, L- and Plus volumes are *not* hardness witnesses: they are satisfiable by construction (Section 7.4) and merely demonstrate that the engine works.

NP-hardness (conjectured). Packing a region with an *arbitrary* tile set is NP-complete [1, 2], and tiling with a *fixed* set of simple tiles is NP-complete as well [3]. This strongly suggests our fixed-8-type problem is NP-hard, but a complete reduction is left as future work. We therefore make no worst-case running-time claim; the timing of Section 7.3 measures one structured, easy family only.

7.2 Configuration Space and Grounding Size

The following counts the size of the naive configuration space (equivalently, the number of ground placement atoms the solver must reason over). We stress that this is *not* a bound on running time — much as the $N!$ permutations of a list do not imply that sorting takes factorial time. It only indicates how large the unpruned space is, motivating why structure and propagation matter. A rough upper bound on the number of joint placements is:

$$\text{Search space} \lesssim \underbrace{(32N \times 24)^{8N}}_{\text{(positions} \times \text{orientations) per piece, over all } 8N \text{ pieces}} \div \underbrace{(N!)^8}_{\text{interchangeable copies}}$$

The factors are read as follows. Each of the $8N$ pieces is placed by choosing an anchor cell (at most $32N$ choices, the cell count) and an orientation. A single piece has at most 24 orientations — the maximum over the eight types, attained by the fully asymmetric L (Table 1); the other types have fewer (3 to 12). Hence each piece has at most $32N \times 24$ placements, and an independent choice for all $8N$ pieces gives the $(32N \times 24)^{8N}$ numerator. Note that the constant 86 used elsewhere is the *sum* of orientations over the eight *types* ($3+12+24+8+3+12+12+12$), not the number of orientations of any one piece, so 24 — not 86 — is the correct per-piece factor here. The denominator $(N!)^8$ divides out the symmetry among the N interchangeable copies of each of the 8 types.

This is only an upper bound on the unpruned space. In practice Clingo’s unit propagation eliminates the vast majority of it: placing one piece immediately constrains where the others can go, often reducing the effective branching factor to near 1 per decision.

The quantity the grounder actually materialises is much smaller. The choice rule generates one ground `position/5` atom per (piece, orientation, anchor) triple, i.e. $\sum_{\text{pieces}} (32N \times r_{\text{type}})$. Summed over the N copies of all eight types this is exactly $32N \times 86 \times N = 2752 N^2$ atoms (verified by grounding with Clingo):

■ **Table 5** Exact ASP grounding size as a function of N : number of ground `position/5` atoms, $2752 N^2$ (confirmed by grounding).

N	Cells ($32N$)	Pieces ($8N$)	Ground <code>position/5</code> atoms ($2752 N^2$)
1	32	8	2 752
2	64	16	11 008
3	96	24	24 768
4	128	32	44 032
7	224	56	134 848

7.3 Empirical Timing: $N = 1$ to 7

Table 6 and Figure 10 report measured Clingo solve times for the T-shape and L-shape at all N from 1 to 7, at maximum difficulty (0 hints), averaged over 3 random seeds.

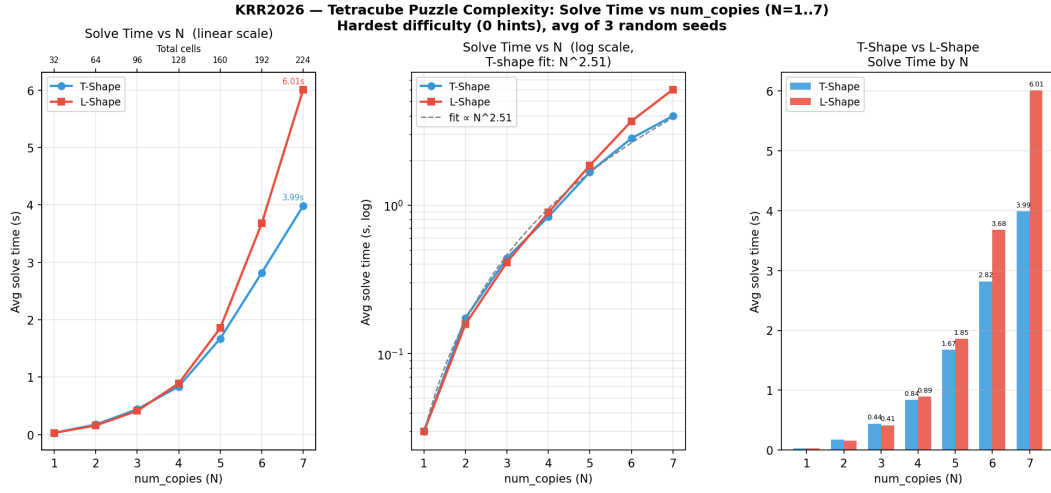
7.4 Scaling Analysis

Several key observations emerge from the timing data:

1. **The timed instances are easy by construction.** Every shape here is a fixed 16-cell XY footprint extruded to depth $2N$ (a *slab*). Such instances are *decomposable*: if the

■ **Table 6** Average solve time (seconds) for T-shape and L-shape, $N = 1$ to $N = 7$. Difficulty: 0 hints (all pieces hidden). Averaged over seeds 43, 44, 45.

N	Cells	Pieces	T-Shape (s)	L-Shape (s)	Slowdown (T)
1	32	8	0.030	0.030	$1\times$
2	64	16	0.174	0.158	$5.8\times$
3	96	24	0.441	0.413	$14.7\times$
4	128	32	0.837	0.892	$27.9\times$
5	160	40	1.673	1.855	$55.8\times$
6	192	48	2.819	3.679	$93.9\times$
7	224	56	3.986	6.012	$132.9\times$



■ **Figure 10** Solve time vs num_copies for T-shape (blue) and L-shape (red). **Left:** linear scale shows the absolute growth from 30 ms to ~6s over $N = 1$ –7. **Centre:** log scale reveals a power law; fitting gives $t \approx c \cdot N^{2.5}$ for the T-shape. **Right:** side-by-side bar chart comparing the two shapes at each N .

depth-2 base ($N = 1$) is packable, then the depth- $2N$ volume is packable by stacking N vertically-translated copies of the base solution (copy k occupying $z \in \{2k, 2k + 1\}$; every tetracube, including the $2 \times 2 \times 2$ Pyramid, fits within two layers). Hence satisfiability at $N = 1$ *implies* satisfiability at all N , constructively and in linear time. The measured $t(N) \approx c \cdot N^{2.5}$ therefore characterises Clingo’s behaviour on a structured, solution-dense family — it is *not* a worst-case bound for the general problem of Section 7.1. (Clingo does not exploit the decomposition; it solves the monolithic exact cover, and the interchangeable copies even enlarge the grounding, so the polynomial growth reflects abundance of solutions plus strong propagation rather than intrinsic ease for the solver.)

- 2. Slab vs. general volumes.** The slab construction is special precisely because it *guarantees* a yes-instance. Removing that structure changes the picture sharply: Table 7 shows that freely-grown 3D polycubes of the same sizes are almost never packable, and that even *deciding* them becomes slower and more variable with N . The slab timing should thus be read as a best-case, not a representative, scaling law.

- 237 **3. L-shape harder than T-shape at large N .** At $N = 7$, the L-shape takes 6 s vs 4 s for
 238 the T-shape. The L-shape’s asymmetric boundary leaves fewer valid anchor positions for
 239 the longer pieces (I, N, Z), creating harder sub-problems.
- 240 **4. Practical scalability.** All instances up to $N = 7$ (224 cells, 56 pieces) solve in under 10
 241 seconds. This is well within interactive response time even at the largest tested scale.
- 242 **5. Slowdown factor.** From $N = 1$ to $N = 7$, T-shape slows by $133\times$. With $N^{2.5}$ scaling
 243 this is consistent: $7^{2.5}/1^{2.5} \approx 130$.

244 7.5 The General Case: Non-Decomposable 3D Volumes

245 To probe the problem *without* the slab guarantee, we generated free 3D polycubes with the
 246 Eden model (`random_puzzle.py -mode 3d`): each is a connected $32N$ -cell volume grown
 247 in all three axes, so its layers differ and no stacking decomposition exists. For each N we
 248 generated shapes until 5 satisfiable instances were found or 400 attempts were exhausted,
 249 solving each at maximum difficulty (0 hints) with a 30 s timeout (`benchmark_3d.py`).

■ **Table 7** Free 3D polycubes (non-decomposable). Unlike the slab family, these are almost never packable, and the decision cost grows with N .

N	Cells	Attempts	SAT rate	UNSAT mean (s)	Timeouts (> 30 s)
1	32	400	0.5%	0.028	0
2	64	400	0.0%	0.113	0
3	96	400	0.0%	0.399	1

250 Two facts stand out. First, satisfiability is *rare*: out of 400 random 64- and 96-cell
 251 polycubes, none admitted a packing, confirming that the decision problem of Section 7.1 is
 252 genuinely two-sided and predominantly negative. Second, the cost of *refuting* an instance
 253 grows with N ($0.028 \rightarrow 0.113 \rightarrow 0.399$ s mean) and becomes variable — one $N = 3$ instance
 254 exceeded the 30 s timeout. This contrasts sharply with the smooth, always-satisfiable slab
 255 curve of Section 7.3, and is the behaviour one expects of a hard, two-sided problem rather
 256 than an easy structured one.

257 8 Conclusion

258 We have built a complete, shape-agnostic, scalable system for 3D tetracube puzzles using
 259 Answer Set Programming.

260 Key achievements:

- 261 ■ Three non-rectangular volumes (T, L, Plus/Cross) and arbitrary random polycubes are
 262 all solved by one unmodified solver algorithm.
- 263 ■ The `num_copies` parameter scales the puzzle from 32 cells ($N = 1$) to 224 cells ($N = 7$)
 264 with no code changes.
- 265 ■ On the (decomposable) slab family, empirical timing from $N = 1$ to $N = 7$ grows
 266 polynomially, $t \approx c \cdot N^{2.5}$; this is a best-case scaling law, not a worst-case bound, since
 267 these instances are satisfiable by construction and stack from the $N = 1$ solution.
- 268 ■ On free 3D polycubes the picture is the opposite: satisfiability is rare and decision cost
 269 grows with N , evidencing the genuinely two-sided, conjectured-NP-hard general problem.
- 270 ■ All slab solutions at $N = 1$, $N = 2$, $N = 3$ pass 6 independent correctness checks.

271 **Design principle validated:** Separating *shape description* (`cell/3`) from *puzzle logic*
272 (`puzzle.lp`), combined with the cyclic `num_copies` scaling, turns a shape-specific one-shot
273 solver into a general puzzle framework. This demonstrates ASP’s strength: declarative
274 constraints naturally generalise over both geometry and scale.

275 References

- 276 **1** Erik D. Demaine and Martin L. Demaine. Jigsaw puzzles, edge matching, and polyomino
277 packing: Connections and complexity. *Graphs and Combinatorics*, 23(S1):195–208, 2007.
278 doi:10.1007/s00373-007-0713-4.
- 279 **2** Michael R. Garey and David S. Johnson. *Computers and Intractability: A Guide to the Theory*
280 *of NP-Completeness*. W. H. Freeman, 1979.
- 281 **3** Cristopher Moore and John Michael Robson. Hard tiling problems with simple tiles. *Discrete*
282 *& Computational Geometry*, 26(4):573–590, 2001. doi:10.1007/s00454-001-0047-6.